

부록 B. 업계 동향과 더 읽을거리

이 부록만 이 책의 다른 모든 장·부록과 근거의 성격이 다르다. 00_기획안.md §2가 밝혔듯 이 책의 본문은 "시장 조사나 외부 논문을 인용하지 않는다"는 원칙을 지킨다. 모든 주장은 Book-forge-Agent-Evaluator의 실제 소스 코드로만 뒷받침한다. 이 부록은 그 원칙의 유일한 예외다. 지금까지 다룬 메커니즘 (HallucinationDetector·PerformanceMonitor·ThreatSeverityConfig·LLM Judge)이 **업계 전체에서는 어떤 이름으로, 어떤 규모로 다뤄지는가**를 짧게 정리해, Book-forge라는 한 도구를 넘어선 맥락을 얻고 싶은 독자에게 출발점을 준다. 여기 인용된 수치·주장은 각 출처의 것이지 이 책이 직접 검증한 것이 아니다. 원문을 확인하고 싶다면 링크를 직접 따라가라.

B.1 환각 탐지 — Gate C: **HallucinationDetector**가 서 있는 자리

3장 (§3.2)·9장 (§9.2)·11장이 다룬 환각(hallucination)은 Book-forge 하나만의 문제가 아니라 업계 전체가 별도 계층 계층을 두고 씨름하는 문제다. Lakera의 정리에 따르면 환각은 "모델 하나의 결함"이 아니라 "빠진 평가 계층(missing eval layer)"의 문제로 프레임이 옮겨가는 중이며, 해법은 근거 제공(grounding) + 런타임 탐지 + 모델별 리포팅의 조합이라고 본다. Guardrails AI·LangKit·RAGAS 같은 프로덕션 도구가 RAG·불확실성 추정·자기일관성(self-consistency)·가드레일을 조합해 40~96%의 환각 감소를 보고한다는 조사도 있다(Zylos Research).

Book-forge의 **HallucinationDetector** (7·9장)는 이 스펙트럼에서 가장 단순한 축인 **RAG 근거 대조** 하나만 쓴다. 자기일관성(같은 질문을 여러 번 물어 답이 같는지 보는 기법)이나 불확실성 추정(모델이 자기 답에 얼마나 확신하는지 수치화하는 기법)은 Book-forge 어디에도 없다. 이 책이 반복해온 "적게, 정확하게 쓴다" 원칙이 여기서도 그대로 적용된 것으로 읽을 수 있다.

- LLM Hallucinations in 2026: How to Understand and Tackle AI's Most Persistent Quirk (Lakera)
- LLM Hallucination Detection and Mitigation: State of the Art in 2026 (Zylos Research)
- The Complete Guide to LLM & AI Agent Evaluation in 2026 (Adaline)

B.2 에이전트 관측성이 독립 분야가 되다 — **PerformanceMonitor**가 서 있는 자리

2장(§ 2.1)·9장(§ 9.1)이 정리한 Tracker·Config·Gate 구조는 업계에서 "AI 에이전트 관측성(agent observability)"이라 불리는, 최근 별도 분야로 분화한 영역과 같은 문제를 다룬다. 업계 조사에 따르면 에이전트 관측성은 "단일 LLM 호출 로깅"이나 일반적인 APM(Application Performance Monitoring)과 구분되는 별도 실천으로 자리잡았다. 핵심 차이는 "무엇을 출력했는가"가 아니라 "그 결과에 이르기까지 에이전트가 내린 결정의 사슬 전체를 설명할 수 있는가"다(digitalapplied.com). LangSmith·Arize Phoenix·Langfuse·Braintrust 같은 전용 플랫폼이 이 영역의 주요 도구로 꼽히고, OpenTelemetry의 GenAI SIG(Special Interest Group)가 멀티에이전트 시스템을 위한 표준 시맨틱 컨벤션을 만드는 중이라는 보도도 있다.

Book-forge의 **PerformanceMonitor** (9·12장)는 이런 전용 플랫폼이 아니다. 별도 서버나 대시보드 없이 `eval_results/*.json` 파일에 기록하고, `book-forge gate`로 병합해 읽는 것이 전부다. "프로덕션에 상시 배포된 여러 에이전트를 실시간으로 관측하는 도구"와 "책 한 권 분량 프로젝트의 품질을 배치로 점검하는 도구"는 애초에 겨냥하는 문제의 규모가 다르다. Book-forge가 후자를 택했다는 것을 17장(§ 17.2)의 "의도적으로 단순한 설계" 논의와 함께 읽으면 이 선택이 왜 합리적인지 더 분명해진다.

- Top 5 Tools for AI Agent Observability in 2026 (Maxim AI)
- AI Agent Observability 2026: Tracing & Monitoring Stack (Digital Applied)
- AI observability tools: A buyer's guide to monitoring AI agents in production (Braintrust)

B.3 프롬프트 인젝션과 OWASP Top 10 — Gate E· **ThreatSeverityConfig**가 서 있는 자리

8장(§ 8.3)이 다룬 **ThreatSeverityConfig** (외부 RAG 소스의 프롬프트 인젝션 위협)는 업계 표준 위험 분류 체계에서 이름이 있는 항목이다. OWASP(Open Worldwide Application Security Project)의 "LLM 애플리케이션 Top 10"에서 프롬프트 인젝션은 **LLM01**, 즉 3년 연속 1위 위험으로 꼽혔다. 2025년판은 에이전트형 AI의 급성장을 반영해 카테고리를 새로 추가·재편했다. 관련 연구는 RAG 소스를 오염시키는 공격(문서 5개만 정교하게 조작해도 90% 확률로 응답을 조작할 수 있었다는 2026년 1월 연구)과, MCP(아래 B.4) 같은 도구 연결이 열어준 새 공격 표면(도구 오염·간접 인젝션)을 지적한다.

rag_mode=True 인 6개 에이전트 (ChapterDrafter·ReferenceTable·Diagram·Capstone·ModuleReference·Chat)만 **ThreatSeverityConfig** 를 쓰는 이유(8장 § 8.3 — "외부에서 온, 신뢰할 수 없는 콘텐츠를 프롬프트에 직접 섞는 에이전트들")는 이 업계 분류로 보면 정확히 **LLM01(프롬프트 인젝션)이 발생할 수 있는 진입점을 식별한 것**과 같다. OWASP가 권고하는 심층 방어(최소 권한 도구, 입출력 필터링, 고위험 동작에 대한 사람 승인)와 견줘보면, Book-forge의 방어는 그중 "탐지·점수화"(**ThreatSeverityConfig**) 측에 해당한다. "사람 승인"은 6장의 저자 승인 루프가, "최소 권한"은 14장의 **@tool_guard** 가 서로 다른 각도에서 담당한다는 것도 흥미로운 대응 관계다.

- [OWASP Top 10 for LLM Applications 2025 \(공식\)](#)
- [The OWASP Top 10 for LLM Applications \(2025\): Explained Simply](#)
- [Prompt Injection Defense for Production AI Agents: A Complete 2026 Guide \(Maxim AI\)](#)

B.4 Model Context Protocol — Book-forge가 다루지 않는 인접 기술

Anthropic이 2024년 말 공개한 MCP(Model Context Protocol)는 "AI 애플리케이션을 위한 USB-C"에 비유되는, LLM과 외부 도구·데이터 소스를 잇는 개방형 인터페이스 표준이다. 각 LLM·도구 조합마다 커스텀 연동 코드를 짜야 했던 "N×M 문제"를 없애는

것이 목적이며, 2026년 기준 월간 다운로드 1억 건을 넘는 등 업계 표준으로 빠르게 자리 잡았다는 보도가 있다.

Book-forge는 MCP를 쓰지 않는다. `knowledge/sources.py`의 소스 어댑터(PDF·코드 저장소·URL)는 Book-forge 자신이 직접 짰 통합이다(1장 § 1.2). 이것이 결함은 아니다. Book-forge의 소스 유형은 몇 가지로 고정돼 있고 자주 바뀌지 않으므로, 범용 프로토콜을 도입하는 비용이 아직 정당화되지 않는다는 판단으로 읽을 수 있다. 다만 17장 (§ 17.5)이 "이 책이 다루지 않은 열린 항목"을 정직하게 나열한 것과 같은 정신으로, MCP 역시 Book-forge가 **의식적으로 채택하지 않은** 인접 기술이라는 점을 여기 남겨둔다. 도구 연동 지점을 더 늘릴 계획이 생긴다면 재검토할 후보다.

- [MCP \(Model Context Protocol\) Emerges as Key AI Interoperability Standard for Multi-Agent Systems in 2026](#)
- [The 2026-07-28 MCP Specification Release Candidate \(공식 블로그\)](#)

B.5 LLM Judge의 신뢰성 논쟁 — 16장의 옵트인 설계가 서 있는 자리

16장 (§ 16.2)은 Book-forge의 `--enable-llm-judge`가 기본 off이고 명시적으로 켜야 한다는 사실을 다뤘다. 이 설계가 왜 신중한지는 LLM Judge(LLM을 채점자로 쓰는 기법) 자체에 대한 최근 연구를 보면 더 분명해진다. 21개 판정 모델을 9개 제공사에서 뽑아 3개 벤치마크·약 54만 건의 개별 판정으로 검증한 2026년 연구는, 같은 판정 모델이 **한 벤치마크에서 다른 벤치마크로 옮기면 순위가 최대 14계단까지 흔들리고**, 회차별 재현성은 높으면서도 응답 순서에 따른 편향(position bias)이 동시에 존재하는 모순("일관성-편향 역설")을 보고했다. 다른 연구들도 LLM Judge가 응답 길이·문체 같은 표면적 신호에 흔들리는 verbosity bias·self-enhancement bias를 지적한다.

Book-forge가 LLM Judge를 "일부 샘플에만, 옵트인으로" 적용하는 것(16장 § 16.2, `judge_sample_rate=0.2`)은 이런 신뢰성 논쟁을 고려하면 자연스러운 선택으로 보인다. 판정 결과를 전면적으로 신뢰하는 대신, 참고 신호 하나로만 제한적으로 섞어 쓰는 태도다.

- [Reliability without Validity: A Systematic, Large-Scale Evaluation of LLM-as-a-Judge Models Across Agreement, Consistency, and Bias \(arXiv\)](#)

- [Evaluating Scoring Bias in LLM-as-a-Judge \(arXiv\)](#)

이 부록을 어떻게 읽어야 하는가

- **여기 나온 통계·순위는 각 출처 하나의 주장이다.** 이 책 나머지 전체가 지키는 "실제 소스 코드로만 검증한다"는 원칙이 이 부록에는 적용되지 않는다. 인용을 그대로 믿기보다 원문을 확인하는 습관이 필요하다.
- **Book-forgе가 이 동향들을 "따라가지 못하고 있다"는 뜻이 아니다.** B.1·B.2가 보여주듯, 오히려 업계의 정교한(그리고 무거운) 해법 대신 의도적으로 더 단순한 해법을 택한 지점이 많다. 그 선택이 이 책 전체가 반복해온 "이 프로젝트 규모에 필요한 만큼만" 원칙과 일관된다.
- **각 절은 이 책의 특정 장으로 되돌아가는 다리다.** B.1→9·11장, B.2→2·9·12·17장, B.3→8·14장, B.4→1·17장, B.5→16장. 순서대로 읽을 필요는 없다.